DEVELOPER GUIDE

Ceremoney Platform

Core Architecture & Development Specification

Wedding Booking & Planning System

Multi-Event Digital Platform

Phase 1 MVP Development

Version 1.0 | March 2026

Table of Contents

1. Executive Summary

2. System Overview

- 2.1 Platform Vision
- 2.2 Core Features
- 2.3 Target Users

3. Technology Stack

- 3.1 Frontend Architecture
- 3.2 Backend Architecture
- 3.3 Database & Storage
- 3.4 Third-Party Services

4. System Architecture

- 4.1 High-Level Architecture
- 4.2 Application Layers
- 4.3 Service Communication

5. Database Schema

- 5.1 Core Entities
- 5.2 Entity Relationships
- 5.3 Database Design Principles

6. API Architecture

- 6.1 API Design Principles
- 6.2 Authentication & Authorization
- 6.3 Core API Endpoints

7. Module Specifications

- 7.1 User Management Module
- 7.2 Event Management Module
- 7.3 Guest Management Module
- 7.4 Seating Chart Module
- 7.5 Website Builder Module
- 7.6 RSVP Module

7.7 Vendor Directory Module

7.8 Billing & Subscription Module

8. Security Architecture

9. Internationalization

10. File Structure & Organization

11. Development Guidelines

12. Deployment Architecture

13. Development Timeline

1. Executive Summary

The **Ceremoney Platform** is a comprehensive web-based wedding booking and planning system designed to serve as an "All-in-One Planning Command Center" for engaged couples. The platform combines the vendor marketplace capabilities of WeddingWire with the intuitive project management tools of Planning.Wedding, while introducing innovative features like a live website builder, interactive seating charts, and digital entrance management.

> **Project Vision**
>
> Shift from a simple vendor directory to a **tool-first experience** where couples can plan, manage, and execute their entire wedding journey through a single unified platform.

Key Differentiators

- **Multi-Event Support:** Not limited to weddings - supports baptisms, parties, corporate events, and private gatherings
- **Live Website Builder:** Real-time visual editing with instant preview
- **SVG-Based Seating Charts:** Professional-grade drag-and-drop seating with venue layout uploads
- **Digital Entrance Mode:** QR code-based guest check-in with seat finder
- **Printable Asset Engine:** Generate matching stationery (place cards, menus, table numbers)
- **Trilingual Support:** Swedish, English, and Arabic with full RTL/LTR capabilities

Development Scope (Phase 1 MVP)

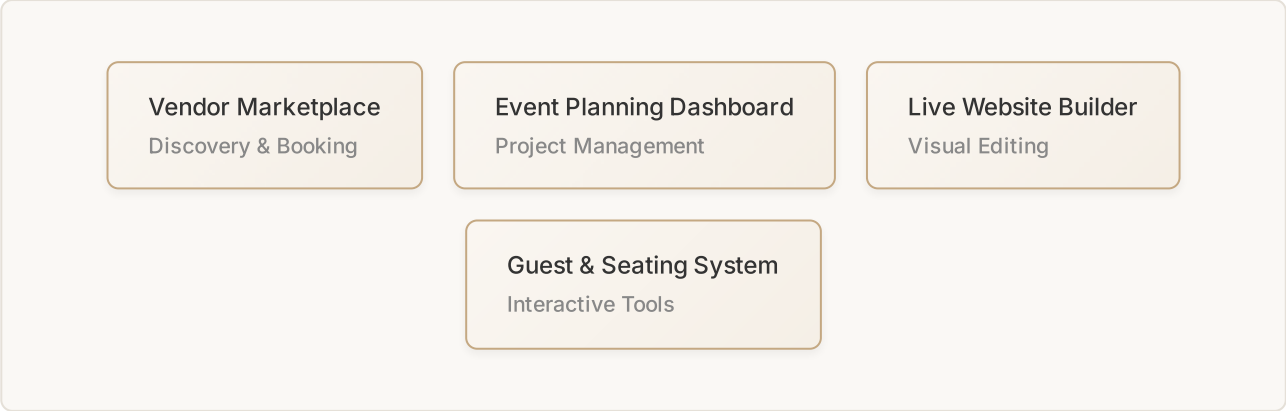
Table 1 Phase 1 Deliverables

Component	Features	Priority
Main Website	Public landing, vendor listings, planning tools overview	Required
User Dashboard	Project overview, guest summary, checklist progress	Required
Guest Management	CRUD operations, CSV import/export, RSVP tracking	Required
Seating Chart	Interactive editor, SVG upload, drag-and-drop assignment	Required
Website Builder	Live editor, templates, RSVP integration	Required
Vendor Directory	Search, profiles, availability calendar	MVP Limited
Billing System	Stripe integration, subscription tiers, Swish/Klarna	Premium

2. System Overview

2.1 Platform Vision

The Ceremoney Platform operates as a **hybrid ecosystem** combining multiple interconnected systems:



2.2 Core Features

Public-Facing Features

- **Landing Page:** Video/interactive sliders showcasing platform tools
- **Vendor Directory:** Category-based search with geo-location filtering
- **Inspiration Hub:** Blog, real weddings, ideas gallery
- **Vendor Profiles:** Service listings, galleries, reviews, availability calendar

User Dashboard Features

- **Project Overview:** Progress tracker, upcoming tasks, budget summary
- **Guest List Management:** Import/export, RSVP tracking, table assignment
- **Seating Chart Editor:** SVG layout upload, drag-and-drop tables
- **Website Builder:** Live editing, template selection, widget management
- **Checklist:** Automated task list based on event date

Event Website Features

- **Responsive Design:** Mobile-first, optimized for all devices
- **Multilingual Toggle:** SE/EN/AR language switching
- **RSVP Form:** Conditional logic based on attendance status
- **Custom Widgets:** Countdown, gallery, location map, wishlist

- **SEO Module:** Custom metadata, Google Analytics integration

2.3 Target Users

Table 2 User Personas

User Type	Primary Goals	Key Features
Engaged Couples	Plan and manage wedding	Dashboard, Guest List, Seating, Website Builder
Event Hosts	Organize baptisms, parties, corporate events	Multi-event support, templates, RSVP
Wedding Planners	Manage multiple client events	Collaborator access, multi-project dashboard
Vendors	Promote services, get bookings	Vendor portal, profile management, calendar
Venue Owners	Manage venue availability and layouts	SVG layout upload, capacity management
Platform Admins	Manage platform operations	Super admin panel, user management, ads

3. Technology Stack

3.1 Frontend Architecture

Next.js 14
React Framework with App Router

React 18
UI Library with Hooks

TypeScript
Type-Safe Development

TailwindCSS
Utility-First Styling

shadcn/ui
Component Library

Zustand
State Management

Additional Frontend Libraries

Table 3 Frontend Dependencies

Library	Purpose	Version
`@dnd-kit`	Drag-and-drop for seating chart	Latest
`react-konva`	Canvas-based SVG rendering	Latest
`react-hook-form`	Form handling and validation	Latest
`zod`	Schema validation	Latest
`react-query`	Server state management	Latest
`next-intl`	Internationalization	Latest
`qrcode.react`	QR code generation	Latest
`jspdf`	PDF generation for exports	Latest

3.2 Backend Architecture

Node.js
Runtime Environment

Express.js
Web Framework

TypeScript
Type-Safe Backend

API Architecture

> **API Pattern: RESTful + tRPC**
>
> - **Public APIs:** RESTful endpoints for external integrations
> - **Internal APIs:** tRPC for type-safe client-server communication
> - **WebSocket:** Real-time updates for collaborative editing

3.3 Database & Storage

PostgreSQL
Primary Database

Prisma ORM
Database Toolkit

Redis
Caching & Sessions

File Storage

- **AWS S3:** Image uploads, PDF exports, venue layouts
- **CloudFront CDN:** Asset delivery optimization
- **Presigned URLs:** Secure direct uploads

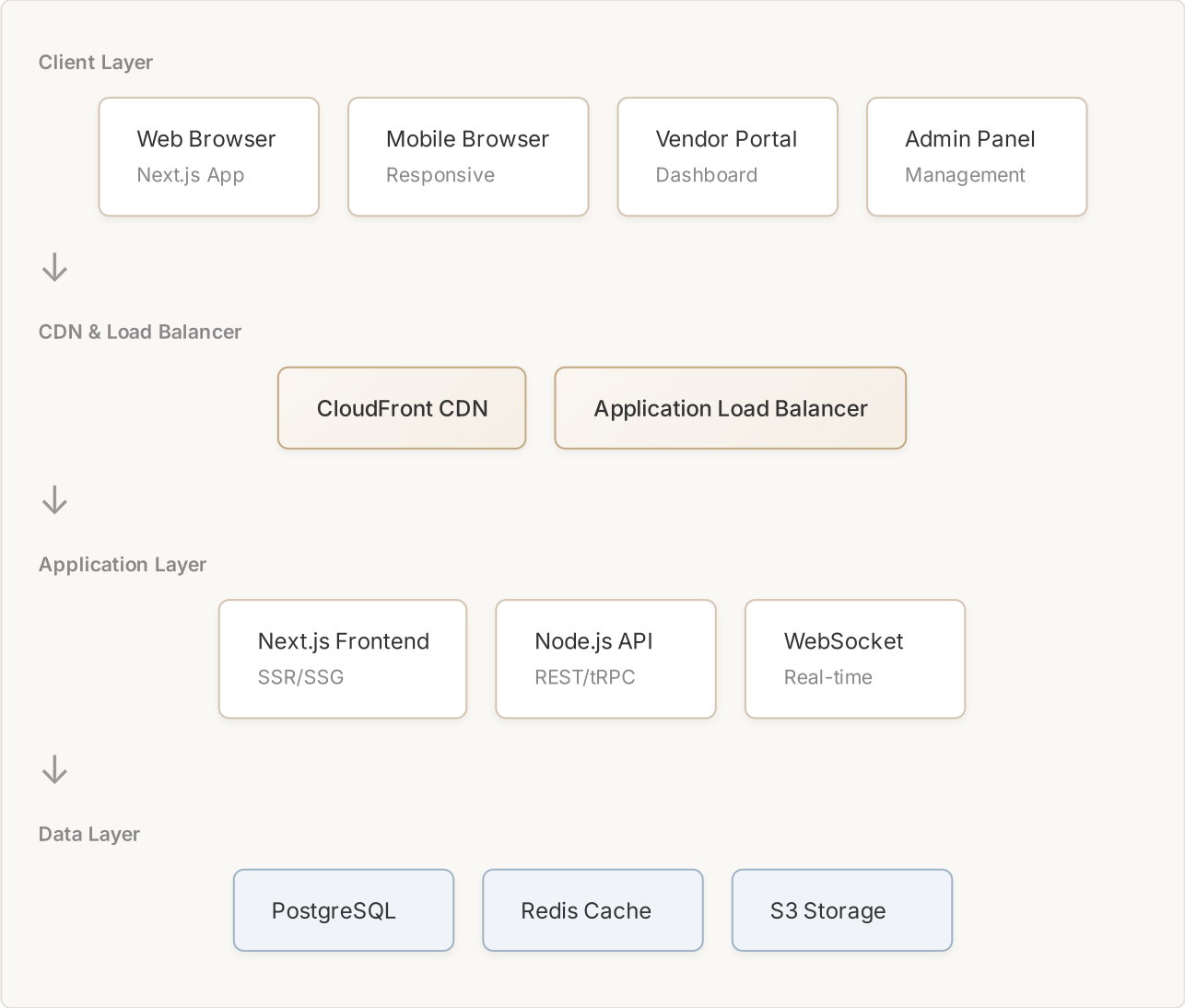
3.4 Third-Party Services

Table 4 Third-Party Integrations

Service	Purpose	Integration Type
Stripe	Payment processing, subscriptions	API + Webhooks
Swish (Sweden)	Local payment method	API
Klarna	Pay Later / Slice It	API
SendGrid/AWS SES	Email delivery	API
Twilio	SMS notifications	API
Google Maps	Location services	JavaScript SDK
Google Analytics	Website analytics	JavaScript SDK

4. System Architecture

4.1 High-Level Architecture



4.2 Application Layers

Presentation Layer

- **Next.js App Router:** Server-side rendering for SEO-critical pages
- **Static Generation:** Pre-built pages for landing, vendor profiles
- **Client Components:** Interactive dashboard, builder, seating chart
- **Route Groups:** Organized by feature (dashboard, admin, public)

Business Logic Layer

- **API Routes:** Next.js API routes for serverless functions
- **Service Layer:** Business logic encapsulated in services
- **Middleware:** Authentication, logging, rate limiting
- **Background Jobs:** Queue-based processing for emails, PDFs

Data Access Layer

- **Prisma Client:** Type-safe database queries
- **Repository Pattern:** Data access abstraction
- **Connection Pooling:** PgBouncer for production

4.3 Service Communication

Communication Patterns

Pattern	Use Case	Implementation
Request-Response	CRUD operations	REST API / tRPC
Real-time	Collaborative editing	WebSocket (Socket.io)
Event-driven	Background processing	Bull Queue + Redis
Webhook	Payment notifications	Stripe webhooks

5. Database Schema

5.1 Core Entities

User	
id	UUID
email	String
password	Hash
firstName	String
lastName	String
role	Enum
createdAt	DateTime

Event	
id	UUID
userId	UUID
type	Enum
name	String
date	DateTime
location	JSON
status	Enum

Guest	
id	UUID
eventId	UUID
firstName	String
lastName	String
email	String
phone	String
rsvpStatus	Enum

Table	
id	UUID
eventId	UUID
name	String
type	Enum
capacity	Int
position	JSON
shape	JSON

SeatAssignment	
id	UUID
guestId	UUID
tableId	UUID
seatNumber	Int
createdAt	DateTime

Website	
id	UUID
eventId	UUID
subdomain	String
customDomain	String
theme	JSON
isPublished	Boolean

5.2 Entity Relationships

```
// Prisma Schema Overview

model User {
  id        String   @id @default(uuid())
  email     String   @unique
  password  String
  firstName String
  lastName  String
  role      UserRole @default(CUSTOMER)
  events    Event[]
  createdAt DateTime @default(now())
}

model Event {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  type        EventType
  name        String
  date        DateTime
  location    Json?
  guests      Guest[]
  tables      Table[]
  website     Website?
  createdAt   DateTime  @default(now())
}

model Guest {
  id           String     @id @default(uuid())
  eventId      String
   event          Event        @relation(fields: [eventId], references: [id], onDelete:
Cascade)
  firstName    String
  lastName     String
  email        String?
  phone        String?
  rsvpStatus   RSVPStatus @default(PENDING)
  guestCount   Int        @default(1)
  dietaryInfo  Json?
  seatAssignment SeatAssignment?
}

model Table {
  id        String         @id @default(uuid())
  eventId   String
   event      Event            @relation(fields: [eventId], references: [id], onDelete:
Cascade)
  name      String
  type      TableType
  capacity  Int
  position  Json           // { x: number, y: number }
```

```
// Prisma Schema Overview

model User {
  id        String   @id @default(uuid())
  email     String   @unique
  password  String
  firstName String
  lastName  String
  role      UserRole @default(CUSTOMER)
  events    Event[]
  createdAt DateTime @default(now())
}

model Event {
  id          String    @id @default(uuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  type        EventType
  name        String
  date        DateTime
  location    Json?
  guests      Guest[]
  tables      Table[]
  website     Website?
  createdAt   DateTime  @default(now())
}

model Guest {
  id           String     @id @default(uuid())
  eventId      String
   event          Event        @relation(fields: [eventId], references: [id], onDelete:
Cascade)
  firstName    String
  lastName     String
  email        String?
  phone        String?
  rsvpStatus   RSVPStatus @default(PENDING)
  guestCount   Int        @default(1)
  dietaryInfo  Json?
  seatAssignment SeatAssignment?
}

model Table {
  id        String         @id @default(uuid())
  eventId   String
   event      Event            @relation(fields: [eventId], references: [id], onDelete:
Cascade)
  name      String
  type      TableType
  capacity  Int
  position  Json           // { x: number, y: number }
  shape     Json           // { width: number, height: number, rotation: number }
```

5.3 Database Design Principles

Design Guidelines

- **Soft Deletes:** Use `deletedAt` fields instead of hard deletes for audit trails
- **JSON Fields:** Store flexible data (themes, positions) as JSON for schema evolution
- **Indexes:** Index frequently queried fields (email, eventId, rsvpStatus)
- **Enums:** Use database enums for status fields with limited values
- **Timestamps:** Always include `createdAt` and `updatedAt`

6. API Architecture

6.1 API Design Principles

- **RESTful Conventions:** Standard HTTP methods (GET, POST, PUT, PATCH, DELETE)
- **Resource-Based URLs:** `/api/events/{id}/guests`
- **Consistent Response Format:** `{ success: boolean, data: any, error?: string }`
- **Versioning:** URL-based versioning `/api/v1/`
- **Pagination:** Cursor-based for large lists

6.2 Authentication & Authorization

Authentication Flow

```
// JWT-based Authentication

1. User logs in with email/password
2. Server validates credentials
3. Server issues JWT access token (15 min expiry)
4. Server issues refresh token (7 day expiry)
5. Client stores tokens in httpOnly cookies
6. Client includes access token in Authorization header
7. Server validates token on each request
8. Refresh token used to get new access token
```

Authorization Roles

Table 5 User Roles & Permissions

Role	Permissions
`CUSTOMER`	Manage own events, guests, websites
`VENDOR`	Manage vendor profile, availability, leads
`COLLABORATOR`	Access specific event (read-only or edit)
`ADMIN`	Full platform management
`SUPER_ADMIN`	System-level access, user management

6.3 Core API Endpoints

POST `/api/v1/auth/login`

Authenticate user and return JWT tokens

Request Body:

```
{ "email": "string", "password": "string" }
```

GET `/api/v1/events`

List all events for authenticated user

Query Parameters:

```
?page=1&limit=10&status=active
```

POST `/api/v1/events`

Create a new event

Request Body:

```
{
  "type": "WEDDING",
  "name": "Emma & John's Wedding",
  "date": "2026-08-15T16:00:00Z",
  "location": { "venue": "Grand Hotel", "address": "..." }
}
```

GET `/api/v1/events/{eventId}/guests`

Get all guests for an event

POST `/api/v1/events/{eventId}/guests`

Add a new guest to event

POST `/api/v1/events/{eventId}/guests/import`

Bulk import guests from CSV/Excel

Request:

```
Content-Type: multipart/form-data
file: CSV or XLSX file
```

GET `/api/v1/events/{eventId}/tables`

Get seating layout with tables and assignments

POST `/api/v1/events/{eventId}/tables`

Create a new table

POST `/api/v1/events/{eventId}/seat-assignments`

Assign guest to a seat

Request Body:

```
{
  "guestId": "uuid",
  "tableId": "uuid",
  "seatNumber": 1
}
```

POST `/api/v1/rsvp/submit`

Submit RSVP response (public endpoint)

Request Body:

```
{
  "invitationToken": "string",
  "firstName": "string",
  "lastName": "string",
  "attendanceStatus": "ATTENDING",
  "email": "string",
  "guestCount": 2,
  "dietaryRestrictions": ["VEGETARIAN"]
}
```

GET `/api/v1/vendors`

Search vendors with filters

Query Parameters:

```
?category=PHOTOGRAPHER&location=Stockholm&date=2026-08-15
```

7. Module Specifications

7.1 User Management Module

U Authentication

JWT-based auth with email/password, social login (Google), password reset

`next-auth`, `bcrypt`

P User Profiles

Profile management, avatar upload, notification preferences

`AWS S3`, `Sharp`

Key Features

- Registration with email verification
- Login with JWT tokens (access + refresh)
- Password reset via email
- Social login (Google OAuth)
- Profile picture upload and cropping
- Multi-factor authentication (future)

7.2 Event Management Module

Event Types

- **Wedding:** Ceremony + Reception, couple names, registry
- **Baptism:** Godparents, blessings, religious elements
- **Party:** Birthday, anniversary, general celebration
- **Corporate:** Company events, conferences

Event Creation Flow

```
1. User clicks "Create Event"
2. Select Event Type (Wedding/Baptism/Party/Corporate)
3. Enter Event Details (name, date, location)
4. Add Host Names
5. Select Languages (SE/EN/AR)
6. Upload Logo (optional)
7. Choose Color Theme (solid or gradient)
8. Event created → Redirect to Dashboard
```

7.3 Guest Management Module

Core Features

- **Add Guest:** Manual entry with all fields
- **Import:** CSV/Excel bulk import with template
- **Export:** PDF/XLS download for external use
- **RSVP Tracking:** Real-time status updates
- **Group Management:** Family groups, plus-ones
- **Tags:** Custom categorization

Guest Data Model

```
interface Guest {
  id: string;
  eventId: string;
  firstName: string;
  lastName: string;
  email?: string;
  phone?: string;
  rsvpStatus: 'PENDING' | 'ATTENDING' | 'NOT_ATTENDING' | 'MAYBE';
  guestCount: number;
  dietaryRestrictions: string[];
  specialRequests?: string;
  tableId?: string;
  seatNumber?: number;
  invitationSent: boolean;
  invitationToken: string;
  tags: string[];
}
```

7.4 Seating Chart Module

> **Technical Implementation**
>
> **Canvas-based rendering** using Konva.js for smooth drag-and-drop interactions. SVG venue layouts can be uploaded as background layers.

Features

- **Table Types:** Round, rectangular, banquet, custom shapes
- **Drag-and-Drop:** Move tables and guests on canvas
- **SVG Upload:** Import venue floor plans

- **Layout Import/Export:** Backup and restore layouts
- **Entrance Search:** Find guests by name/seat for check-in
- **PDF Export:** Print-ready seating charts

Table Shapes

Table 6 Supported Table Types

Type	Shape	Default Capacity
ROUND	Circle	8-10 seats
RECTANGULAR	Rectangle	6-8 seats
BANQUET	Long rectangle	10-20 seats
HEAD	Special head table	Variable
CUSTOM	User-defined	Custom

7.5 Website Builder Module

Builder Architecture

- **Live Preview:** Real-time WYSIWYG editing
- **Component-based:** Modular widget system
- **Template Engine:** Event-type specific templates
- **Theme System:** Colors, fonts, gradients

Available Widgets

Table 7 Builder Widgets

Widget	Description	Event Types
Header	Hero image, couple names, date	All
Countdown	Live countdown to event	All
Our Story	Timeline of relationship	Wedding
Location	Map, directions, parking info	All
People List	Wedding party, godparents	Wedding, Baptism
Gallery	Photo gallery with lightbox	All

Wishlist	Gift registry links	Wedding, Baptism
RSVP	Interactive RSVP form	All
Schedule	Event timeline	All
Accommodation	Hotel block info	Wedding

7.6 RSVP Module

RSVP Form Logic

```
// Conditional Field Display

IF attendanceStatus === "NOT_ATTENDING":
  SHOW: firstName, lastName, phoneNumber (optional)
  HIDE: All other fields

IF attendanceStatus === "ATTENDING":
  SHOW: All fields
  - email (required)
  - guestCount (slider, required)
  - highChairNeeded (yes/no)
  - dietaryRestrictions (checkboxes)
  - specialMealRequest (textarea)
  - needsAccommodation (yes/no)
  - transportationType (dropdown)
  - songRequest (text)
  - messageToHost (textarea)
  - customQuestions (dynamic)
```

Invitation Delivery Methods

- **Email:** Personalized email with RSVP link
- **Unique Link:** Shareable URL per guest/group
- **QR Code:** Generated QR for physical invitations

7.7 Vendor Directory Module

Vendor Profile Structure

- **Basic Info:** Name, category, location, contact
- **Gallery:** Portfolio images
- **Services:** Offered packages and pricing
- **Reviews:** Customer ratings and testimonials
- **Availability:** Calendar integration

- **Social Links:** Instagram, website

Search & Filter

- Category filter (Photographer, Venue, Caterer, etc.)
- Location-based search with geo-location
- Availability filter by event date
- Price range filter
- Rating filter

7.8 Billing & Subscription Module

Subscription Tiers

Table 8 Subscription Plans

Plan	Features	Price
Basic (Free)	Website (subdomain), Ads enabled, Basic RSVP	Free
Premium	Everything in Basic + Custom domain, Seating Chart, Guest List, No ads	299 SEK/mo
Elite	Everything in Premium + Printable assets, QR Entrance, Collaborators	499 SEK/mo

Payment Methods

- **Global:** Credit Cards, Apple Pay, Google Pay, PayPal
- **Sweden:** Swish, Klarna (Pay Later, Slice It)
- **Invoicing:** PDF receipts with Swedish VAT (Moms)

8. Security Architecture

Authentication Security

- **Password Hashing:** bcrypt with salt rounds 12
- **JWT Security:** Short-lived access tokens (15 min), secure refresh tokens
- **HttpOnly Cookies:** Prevent XSS token theft
- **CSRF Protection:** Double-submit cookie pattern
- **Rate Limiting:** Prevent brute force attacks

Data Protection

- **Encryption at Rest:** Database encryption for sensitive fields
- **Encryption in Transit:** TLS 1.3 for all communications
- **PII Handling:** GDPR-compliant data processing
- **Data Retention:** Automated cleanup policies

API Security

- **Input Validation:** Zod schemas for all inputs
- **SQL Injection Prevention:** Parameterized queries via Prisma
- **XSS Prevention:** Output encoding, CSP headers
- **Authorization Checks:** Resource-level access control

> **Security Checklist**
>
> - All API endpoints require authentication (except public RSVP)
> - Validate user owns requested resource
> - Sanitize all user inputs
> - Use prepared statements for database queries
> - Implement proper CORS policies
> - Set security headers (HSTS, X-Frame-Options, etc.)

9. Internationalization

Supported Languages

Table 9 Language Support

Language	Code	Direction	Font
Swedish	sv	LTR	Inter
English	en	LTR	Inter
Arabic	ar	RTL	IBM Plex Sans Arabic

Implementation Strategy

```
// next-intl Configuration

// Middleware for locale detection
export const config = {
  locales: ['en', 'sv', 'ar'],
  defaultLocale: 'sv'
};

// RTL Support
const rtlLocales = ['ar'];
const isRTL = rtlLocales.includes(locale);

// Apply dir attribute
<html dir={isRTL ? 'rtl' : 'ltr'} lang={locale}>

// Tailwind RTL plugin
// npm install tailwindcss-rtl
module.exports = {
  plugins: [require('tailwindcss-rtl')]
}
```

Translation Structure

```
/messages
  /en
    common.json
    auth.json
    dashboard.json
    guestList.json
    seating.json
    builder.json
  /sv
    common.json
    auth.json
    ...
  /ar
    common.json
    auth.json
    ...
```

10. File Structure & Organization

```
/apps
   /web
      /app
         /(public)
         # Public-facing pages
         page.tsx
         layout.tsx
         /(dashboard)
         # Authenticated dashboard
         /projects
         /events/[eventId]
         /guests
         /seating
         /website
         /(admin)
         # Admin panel
         /api
         # API routes
      /components
         # Shared components
      /hooks
         # Custom React hooks
      /lib
         # Utilities, constants
      /stores
         # Zustand stores
      /types
         # TypeScript types
      /styles
         # Global styles
      /messages
         # i18n translations
/packages
   /database
         # Prisma schema, client
   /api-client
         # Shared API client
   /ui
         # Shared UI components
/services
   # Backend services (if separate)
/infrastructure
   # Terraform, Docker, K8s
```

11. Development Guidelines

Coding Standards

- **TypeScript:** Strict mode enabled, no `any` types
- **ESLint:** Next.js recommended rules + custom rules
- **Prettier:** Consistent code formatting
- **Component Naming:** PascalCase for components, camelCase for functions
- **File Naming:** kebab-case for files, index.ts for barrel exports

Git Workflow

```
// Branch Strategy

main            - Production releases
develop         - Integration branch
feature/*       - New features
bugfix/*        - Bug fixes
hotfix/*        - Emergency production fixes
release/*       - Release preparation

// Commit Convention
feat: add guest import functionality
fix: resolve RSVP form validation bug
docs: update API documentation
style: fix seating chart button alignment
refactor: simplify guest list filtering
test: add unit tests for auth service
chore: update dependencies
```

Testing Strategy

Table 10 Testing Levels

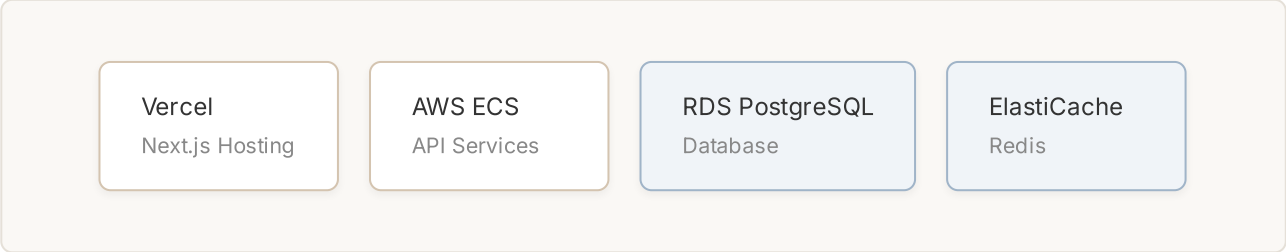
Type	Tool	Coverage Target
Unit Tests	Vitest + React Testing Library	80%
Integration Tests	Vitest + MSW	60%
E2E Tests	Playwright	Critical paths

Performance Guidelines

- **Image Optimization:** Use Next.js Image component with WebP
- **Code Splitting:** Dynamic imports for heavy components
- **Bundle Size:** Monitor with @next/bundle-analyzer
- **Core Web Vitals:** LCP < 2.5s, FID < 100ms, CLS < 0.1
- **Caching:** Implement proper HTTP caching headers

12. Deployment Architecture

Production Environment



CI/CD Pipeline

```
1. Developer pushes to feature branch
2. GitHub Actions runs lint, test, build
3. On PR: Deploy to Preview environment
4. Code review and approval
5. Merge to develop: Deploy to Staging
6. Run E2E tests on Staging
7. Merge to main: Deploy to Production
8. Post-deployment smoke tests
```

Environment Variables

```
# Required Environment Variables

# Database
DATABASE_URL="postgresql://user:pass@host:5432/db"

# Authentication
JWT_SECRET="your-secret-key"
NEXTAUTH_SECRET="your-nextauth-secret"
NEXTAUTH_URL="https://ceremoney.se"

# AWS
AWS_ACCESS_KEY_ID=""
AWS_SECRET_ACCESS_KEY=""
AWS_S3_BUCKET="ceremoney-assets"
AWS_REGION="eu-north-1"

# Stripe
STRIPE_PUBLIC_KEY="pk_live_..."
STRIPE_SECRET_KEY="sk_live_..."
STRIPE_WEBHOOK_SECRET="whsec_..."

# Email
SENDGRID_API_KEY=""
EMAIL_FROM="noreply@ceremoney.se"

# Google
GOOGLE_CLIENT_ID=""
GOOGLE_CLIENT_SECRET=""
GOOGLE_MAPS_API_KEY=""
```

Monitoring & Logging

- **Error Tracking:** Sentry for error monitoring
- **Analytics:** Vercel Analytics + Google Analytics
- **Logging:** Structured logging with Pino
- **Uptime:** Pingdom or UptimeRobot

13. Development Timeline

Phase 1 MVP (18 Weeks)

Weeks 1-2
Architecture & Database Design
System architecture, database schema, API design, project setup

Weeks 3-6
Frontend Development
Landing page, dashboard UI, component library, responsive design

Weeks 7-10
Backend Development
API implementation, authentication, database integration

Weeks 11-14
Builder & Seating Engine
Website builder, drag-and-drop seating, SVG integration

Weeks 15-16
Printable Assets & QR Logic
PDF generation, QR codes, entrance mode

Weeks 17-19
Testing & QA
Unit tests, integration tests, E2E tests, bug fixes

Week 20
Deployment
Production deployment, performance optimization, monitoring setup

Milestone Deliverables

Table 11 Development Milestones

Milestone	Deliverables	Week
M1: Foundation	Project setup, database schema, auth system	4
M2: Core Features	Guest management, basic seating, event creation	8
M3: Builder	Website builder with templates, RSVP forms	12
M4: Advanced	SVG seating, PDF exports, QR codes	16
M5: Launch Ready	All features complete, tested, deployed	20

Development Notes

- Weekly progress reports required
- Live review sessions every 10 days
- UX/UI designs must be approved before coding
- Performance optimization is non-negotiable
- All code must pass review before merge